ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

“НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО”

Факультет Прикладной информатики

Дисциплина: «Алгоритмы и структуры данных»

ОТЧЁТ

по лабораторной работе №2
Вариант 4

Выполнил:
Студент группы K3360
Демакова Ксения Александровна
Преподаватель:
Ромакина Оксана Михайловна

Санкт-Петербург
2026

Задание №4. Простейший неявный ключ (BST по неявному ключу)

Задание

В этой задаче вам нужно написать BST по **неявному** ключу и отвечать им на запросы:

- «+ x» – добавить в дерево x (если x уже есть, ничего не делать).
- «? k» – вернуть k-й по возрастанию элемент.

- **Формат ввода / входного файла (input.txt).** В каждой строке содержится один запрос. Все x - целые числа, количество запросов N не указано в начале, не более 300 000. Гарантируется, что все x выбраны равномерным распределением.
- Случайные данные! Не нужно ничего специально балансировать.
- **Ограничения на входные данные.** $1 \le x \le 10^9$, $1 \le N \le 300000$, в запросах «? k», число k от 1 до количества элементов в дереве.
- **Формат вывода / выходного файла (output.txt).** Для каждого запроса вида «? k» выведите в отдельной строке ответ.
- Ограничение по времени. 2 сек.
- Ограничение по памяти. 256 мб.
- Пример:

input.txt	output.txt
+ 1	1
+ 4	3
+ 3	4
+ 3	3
? 1	
? 2	
? 3	
+ 2	
? 3	

Описание решения

Для решения я реализовала обычное двоичное дерево поиска (BST). Для каждого узла поддерживается размер его поддерева: сравниваем k с размером левого поддерева, и в зависимости от результата либо возвращаем текущий элемент, либо спускаемся в левое или правое поддерево.
Вставка выполняется рекурсивно: спускаемся по дереву, сравнивая ключи, и при необходимости создаем новый узел. После вставки обновляем размеры поддеревьев на пути подъёма. Дубликаты игнорируются, так как при равенстве ключей рекурсия не продолжается.
Поиск k-го элемента реализован рекурсивно через функцию `find_k`, которая использует размеры левого поддерева для определения позиции элемента

Исходный код

```
import sys

sys.stdin = open('input.txt', 'r')
values = []
sizes = []
left = []
right = []


def new_node(x):
    values.append(x)
    sizes.append(1)
    left.append(-1)
    right.append(-1)
    return len(values) - 1


def get_size(ind):
    if ind == -1:
        return 0
    return sizes[ind]


def update_size(ind):
    if ind != -1:
        sizes[ind] = 1 + get_size(left[ind]) + get_size(right[ind])


def insert(ind, x):
    if ind == -1:
        return new_node(x)

    if x < values[ind]:
        left[ind] = insert(left[ind], x)
    elif x > values[ind]:
        right[ind] = insert(right[ind], x)

    update_size(ind)
    return ind


def find_k(ind, k):
    left_size = get_size(left[ind])
    right_size = get_size(right[ind])

    if k == left_size + 1:
        return values[ind]
```

```
    elif k <= left_size:
        return find_k(left[ind], k)
    else:
        return find_k(right[ind], k - left_size - 1)


def main():
    root = -1

    lines = sys.stdin.read().splitlines()
    for line in lines:
        if not line.strip():
            continue

        parts = line.split()
        operation = parts[0]
        x = int(parts[1])

        if operation == '+':
            root = insert(root, x)
        elif operation == '?':
            answer = find_k(root, x)
            print(answer)


if __name__ == '__main__':
    main()
```

Тесты

input.txt

```
1  + 1
2  + 4
3  + 3
4  + 3
5  ? 1
6  ? 2
7  ? 3
8  + 2
9  ? 3
```

```
1
3
4
3
```

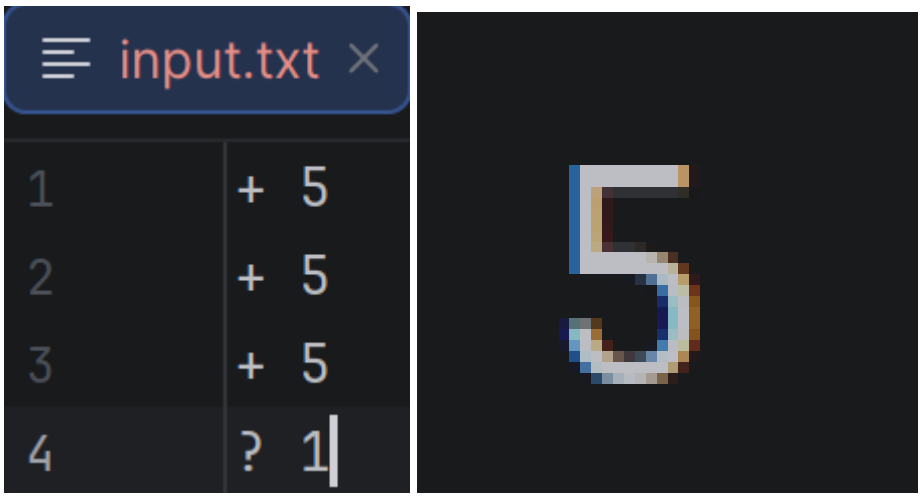
Тест 1

input.txt

```
1  + 5
2  + 3
3  + 7
4  ? 1
5  ? 2
6  ? 3
```

```
3
5
7
```

Тест 2



Tecm 3

Задание №9. Удаление поддеревьев

Задание

Дано некоторое двоичное дерево поиска. Также даны запросы на удаление из него вершин, имеющих заданные ключи, причем вершины удаляются целиком вместе со своими поддеревьями.

После каждого запроса на удаление выведите число оставшихся вершин в дереве.

В вершинах данного дерева записаны ключи – целые числа, по модулю не превышающие 10^9. Гарантируется, что данное дерево является двоичным деревом поиска, в частности, для каждой вершины дерева V выполняется следующее условие:

- все ключи вершин из левого поддерева меньше ключа вершины V;
- все ключи вершин из правого поддерева больше ключа вершины V.

Высота дерева не превосходит 25, таким образом, можно считать, что оно сбалансировано.

- **Формат ввода / входного файла (input.txt).** Входной файл содержит описание двоичного дерева и описание запросов на удаление.

  В первой строке файла находится число N – число вершин в дереве. В последующих N строках файла находятся описания вершин дерева. В $(i+1)$-ой строке файла $(1 \le i \le N)$ находится описание i-ой вершины, состоящее из трех чисел K_i, L_i, R_i, разделенных пробелами – ключа K_i в i-ой вершине, номера левого L_i ребенка i-ой вершины $(i < L_i \le N$ или $L_i = 0$, если левого ребенка нет) и номера правого R_i ребенка i-ой вершины $(i < R_i \le N$ или $R_i = 0$, если правого ребенка нет).

  Все ключи различны. Гарантируется, что данное дерево является деревом поиска.

  В следующей строке находится число M – число запросов на удаление. В следующей строке находятся M чисел, разделенных пробелами – ключи, вершины с которыми (вместе с их поддеревьями) необходимо удалить. Все эти числа не превосходят 10^9 по абсолютному значению. Вершина с таким ключом не обязана существовать в дереве – в этом случае дерево изменять не требуется. Гарантируется, что корень дерева никогда не будет удален.

- **Ограничения на входные данные.** $1 \le N \le 2 \cdot 10^5$, $|K_i| \le 10^9$, $1 \le M \le 2 \cdot 10^5$
- **Формат вывода / выходного файла (output.txt).** Выведите M строк. На i-ой строке требуется вывести число вершин, оставшихся в дереве после выполнения i-го запроса на удаление.
- **Ограничение по времени. 2 сек.**
- Ограничение по памяти. 256 мб.
- Пример:

input.txt	output.txt
6 -2 0 2 8 4 3 9 0 0 3 6 5 6 0 0 0 0 0 4 6 9 7 8	5 4 4 1

Описание решения

Дерево хранится в массивах key, L, R (индексация с 1). Есть массив alive, отмечающий, какие вершины еще существуют, и счётчик remaining.

Для каждого запроса:
Спускаемся от корня по BST, сравнивая ключи, пока не будет найдена вершина с нужным ключом (или не выйдем за пределы дерева).

Если вершина найдена и жива, обход её поддерева, помечая все вершины как удаленные и подсчитывая их количество. Вычет этого количества из remaining и вывод результата

Исходный код

```
data = open("input.txt", "rb").read().split()

p = 0
n = int(data[p]); p += 1

key = [0]*(n+1)
L = [0]*(n+1)
R = [0]*(n+1)
for i in range(1, n+1):
    key[i] = int(data[p]); p += 1
    L[i] = int(data[p]); p += 1
    R[i] = int(data[p]); p += 1

size = [0]*(n+1)
child = [False]*(n+1)
for i in range(1, n+1):
    if L[i]: child[L[i]] = True
    if R[i]: child[R[i]] = True
root = 1
for i in range(1, n+1):
    if not child[i]:
        root = i
        break

order = []
st = [root]
while st:
    v = st.pop()
    order.append(v)
    if L[v]: st.append(L[v])
    if R[v]: st.append(R[v])
for v in reversed(order):
    size[v] = 1 + (size[L[v]] if L[v] else 0) + (size[R[v]] if R[v]
else 0)

alive = [True]*(n+1)
remaining = n

m = int(data[p]); p += 1
out = []
for _ in range(m):
```

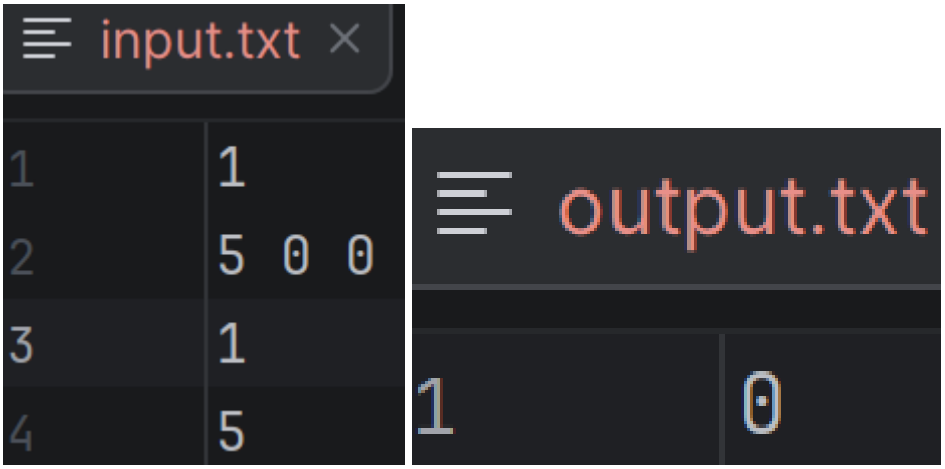
```
    target = int(data[p]); p += 1

    v = root
    while v and alive[v] and key[v] != target:
        v = L[v] if target < key[v] else R[v]

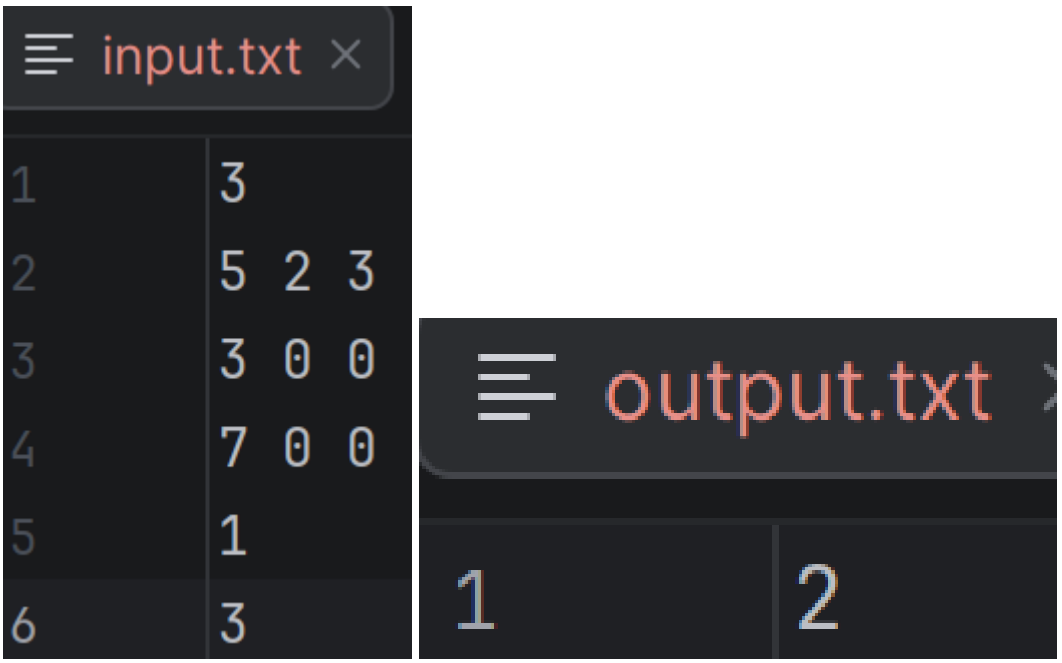
    if v and alive[v]:
        cnt = 0
        st = [v]
        while st:
            u = st.pop()
            if not alive[u]:
                continue
            alive[u] = False
            cnt += 1
            if L[u]: st.append(L[u])
            if R[u]: st.append(R[u])
        remaining -= cnt

    out.append(str(remaining))
open("output.txt", "w").write("\n".join(out))
```

Тесты



Тест 1


input.txt
1 3
2 5 2 3
3 3 0 0
4 7 0 0
5 1
6 3
output.txt
1 2

Тест 2


input.txt
1 5
2 5 2 4
3 3 3 5
4 1 0 0
5 2 0 0
6 7 0 0
7 1
8 3
output.txt
1 2

Тест 3

Задание №15. Удаление из АВЛ-дерева

Задание

Удаление из АВЛ-дерева вершины с ключом X, при условии ее наличия, осуществляется следующим образом:

- путем спуска от корня и проверки ключей находится V – удаляемая вершина;
- если вершина V – лист (то есть, у нее нет детей):
  - удаляем вершину;
  - поднимаемся к корню, начиная с бывшего родителя вершины V, при этом если встречается несбалансированная вершина, то производим поворот.
- если у вершины V не существует левого ребенка:
  - следовательно, баланс вершины равен единице и ее правый ребенок – лист;
  - заменяем вершину V ее правым ребенком;
  - поднимаемся к корню, производя, где необходимо, балансировку.
- иначе:
  - находим R – самую правую вершину в левом поддереве;
  - переносим ключ вершины R в вершину V;
  - удаляем вершину R (у нее нет правого ребенка, поэтому она либо лист, либо имеет левого ребенка, являющегося листом);
  - поднимаемся к корню, начиная с бывшего родителя вершины R, производя балансировку.

Исключением является случай, когда производится удаление из дерева, состоящего из одной вершины - корня. Результатом удаления в этом случае будет пустое дерево.

Указанный алгоритм не является единственно возможным, но мы просим Вас реализовать именно его, так как тестирующая система проверяет точное равенство получающихся деревьев.

- **Формат ввода / входного файла (input.txt).** Входной файл содержит описание двоичного дерева, а также ключа вершины, которую требуется удалить из дерева.

  В первой строке файла находится число N – число вершин в дереве. В последующих N строках файла находятся описания вершин дерева. В $(i+1)$-ой строке файла $(1 \le i \le N)$ находится описание i-ой вершины, состоящее из трех чисел K_i, L_i, R_i, разделенных пробелами – ключа K_i в i-ой вершине, номера левого L_i ребенка i-ой вершины $(i < L_i \le N$ или $L_i = 0$, если левого ребенка нет) и номера правого R_i ребенка i-ой вершины $(i < R_i \le N$ или $R_i = 0$, если правого ребенка нет). Все ключи различны. Гарантируется, что данное дерево является деревом поиска.

  В последней строке содержится число X – ключ вершины, которую требуется удалить из дерева. Гарантируется, что такая вершина в дереве существует.
- **Ограничения на входные данные.** $1 \le N \le 2 \cdot 10^5$, $|K_i| \le 10^9$, $|X| \le 10^9$.
- **Формат вывода / выходного файла (output.txt).** Выведите в том же формате дерево после осуществления операции удаления. Нумерация вершин может быть произвольной при условии соблюдения формата.
- **Ограничение по времени. 2 сек.**
- Ограничение по памяти. 256 мб.
- Пример:

input.txt	output.txt
3 4 2 3 3 0 0 5 0 0 4	2 3 0 2 5 0 0

Описание решения

Спуск от корня до вершины target с ключом X.

Если target - лист: удаляем её, запоминаем родителя как начало подъёма.
Если у target нет левого ребёнка: заменяем её правым ребёнком, поднимаемся от родителя.Иначе: находим самую правую вершину R в левом поддереве, переносим её ключ в target, удаляем R (у неё нет правого ребёнка), поднимаемся от бывшего родителя R. На пути к корню вызов rebalance для каждой вершины, обновляя ссылки у родителей.
Отдельно обрабатывается случай n == 1: вывод 0 (пустое дерево).
После удаления выполняется обход дерева для сбора живых вершин и перенумерации

Исходный код

```
import sys

sys.setrecursionlimit(300000)

data = open("input.txt", "rb").read().split()
p = 0
n = int(data[p]); p += 1

key = [0]*(n+1)
L = [0]*(n+1)
R = [0]*(n+1)
for i in range(1, n+1):
    key[i] = int(data[p]); p += 1
    L[i] = int(data[p]); p += 1
    R[i] = int(data[p]); p += 1
x = int(data[p]); p += 1

child = [False]*(n+1)
for i in range(1, n+1):
    if L[i]: child[L[i]] = True
    if R[i]: child[R[i]] = True
root = 1
for i in range(1, n+1):
    if not child[i]:
        root = i
        break

height = [0]*(n+1)

def h(v):
    return height[v] if v else 0

order = []
st = [root]
while st:
    v = st.pop()
    order.append(v)
    if L[v]: st.append(L[v])
```

```
    if R[v]: st.append(R[v])
for v in reversed(order):
    hl = h(L[v]); hr = h(R[v])
    height[v] = 1 + (hl if hl > hr else hr)

def bal(v):
    return h(R[v]) - h(L[v])

def rotate_right(v):
    u = L[v]
    L[v] = R[u]
    R[u] = v
    height[v] = 1 + max(h(L[v]), h(R[v]))
    height[u] = 1 + max(h(L[u]), h(R[u]))
    return u

def rotate_left(v):
    u = R[v]
    R[v] = L[u]
    L[u] = v
    height[v] = 1 + max(h(L[v]), h(R[v]))
    height[u] = 1 + max(h(L[u]), h(R[u]))
    return u

def rebalance(v):
    height[v] = 1 + max(h(L[v]), h(R[v]))
    b = bal(v)
    if b == 2:
        if bal(R[v]) < 0:
            R[v] = rotate_right(R[v])
        return rotate_left(v)
    if b == -2:
        if bal(L[v]) > 0:
            L[v] = rotate_left(L[v])
        return rotate_right(v)
    return v

v = root
while v and key[v] != x:
    v = L[v] if x < key[v] else R[v]
target = v

def find_parent(v):
    if v == root:
        return 0
    cur = root
    while True:
        if L[cur] == v or R[cur] == v:
```

```
            return cur
        cur = L[cur] if key[v] < key[cur] else R[cur]

if n == 1:
    print(0)
    sys.exit()

if not L[target] and not R[target]:
    R_del = target
    start_parent = find_parent(target)
elif not L[target]:
    par = find_parent(target)
    if par == 0:
        root = R[target]
    elif L[par] == target:
        L[par] = R[target]
    else:
        R[par] = R[target]
    start_parent = par
    R_del = 0
else:
    rr = L[target]
    while R[rr]:
        rr = R[rr]
    key[target] = key[rr]
    R_del = rr
    start_parent = find_parent(rr)

if R_del:
    par = start_parent
    if par == 0:
        root = L[R_del] if L[R_del] else R[R_del]
    elif L[par] == R_del:
        L[par] = L[R_del] if L[R_del] else R[R_del]
    else:
        R[par] = L[R_del] if L[R_del] else R[R_del]

cur = start_parent
while cur:
    par = find_parent(cur)
    new_root = rebalance(cur)
    if par == 0:
        root = new_root
    elif L[par] == cur:
        L[par] = new_root
    else:
        R[par] = new_root
    cur = par
```

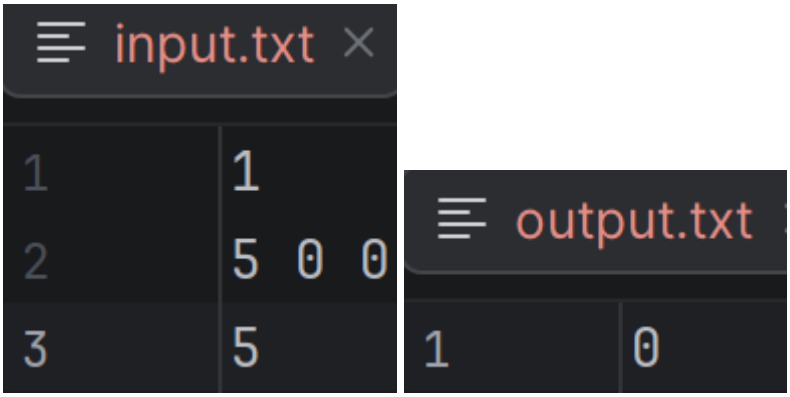
```
alive = []
st = [root]
while st:
    v = st.pop()
    if not v: continue
    alive.append(v)
    if L[v]: st.append(L[v])
    if R[v]: st.append(R[v])

newnum = {}
for i, v in enumerate(alive, 1):
    newnum[v] = i

out = [str(len(alive))]
for v in alive:
    nl = newnum[L[v]] if L[v] else 0
    nr = newnum[R[v]] if R[v] else 0
    out.append(f"{key[v]} {nl} {nr}")

open("output.txt", "w").write("\n".join(out))
```

Тесты



Тест 1

input.txt

```
3
5 2 3
3 0 0
7 0 0
3
```

output.txt

```
2
5 0 2
7 0 0
```

Тест 2

input.txt

```
5
5 2 3
3 4 5
7 0 0
1 0 0
4 0 0
3
```

output.txt

```
4
5 3 2
7 0 0
1 0 4
4 0 0
```

Тест 3

Задание №16. K-й максимум

Задание

Напишите программу, реализующую структуру данных, позволяющую добавлять и удалять элементы, а также находить k-й максимум.

- **Формат ввода / входного файла (input.txt).** Первая строка входного файла содержит натуральное число n – количество команд. Последующие n строк содержат по одной команде каждая. Команда записывается в виде двух чисел c_i и k_i – тип и аргумент команды соответственно. Поддерживаемые команды:
  - +1 (или просто 1): Добавить элемент с ключом k_i.
  - 0 : Найти и вывести k_i-й максимум.
  - -1 : Удалить элемент с ключом k_i.

  Гарантируется, что в процессе работы в структуре не требуется хранить элементы с равными ключами или удалять несуществующие элементы. Также гарантируется, что при запросе ki-го мак- симума, он существует.
- **Ограничения на входные данные.** $n \leq 100000$, $|k_i| \leq 10^9$.
- **Формат вывода / выходного файла (output.txt).** Для каждой команды нулевого типа в выходной файл должна быть выведена строка, содержащая единственное число – k_i-й максимум.
- Ограничение по времени. 2 сек.
- Ограничение по памяти. 512 мб.
- Пример:

input.txt	output.txt
11	7
+1 5	5
+1 3	3
+1 7	10
0 1	7
0 2	3
0 3	
-1 5	
+1 10	
0 1	
0 2	
0 3	

Описание решения

Использовала treap по явному ключу с поддержкой размеров поддеревьев. Каждый узел хранит ключ, приоритет (случайный), ссылки на левого и правого ребёнка и размер поддерева.
Реализованы операции:
split(v, k) - разбивает дерево на два: элементы с ключом < k и >= k;
merge(a, b) - сливает два дерева, сохраняя свойство кучи по приоритетам;
insert(root, k) - разбивает по k, создает узел, сливает обратно;
erase(root, k) — разбивает по k и по k+1, «выбрасывает» среднюю часть, сливает края;
kth_max(root, k) — спуск по дереву: сравниваем k с размером правого поддерева; если k == r+1, возвращаем ключ; если k <= r, идём вправо; иначе уменьшаем k на r+1 и идём влево.
Приоритеты генерируются через random.getrandbits(30) с фиксированным seed для воспроизводимости

Исходный код

```
import sys
import random

sys.setrecursionlimit(300000)

def main():
    data = open("input.txt", "rb").read().split()
    n = int(data[0])
    pos = 1

    random.seed(42)

    key = []
    prio = []
    left = []
    right = []
    size = []

    def new_node(k):
        key.append(k)
        prio.append(random.getrandbits(30))
        left.append(-1)
        right.append(-1)
        size.append(1)
        return len(key) - 1

    def sz(v):
        return size[v] if v != -1 else 0

    def upd(v):
        size[v] = 1 + sz(left[v]) + sz(right[v])

    def split(v, k):
        if v == -1:
            return -1, -1
        if key[v] < k:
            a, b = split(right[v], k)
            right[v] = a
            upd(v)
            return v, b
        else:
            a, b = split(left[v], k)
            left[v] = b
            upd(v)
            return a, v

    def merge(a, b):
```

```
    if a == -1: return b
    if b == -1: return a
    if prio[a] > prio[b]:
        right[a] = merge(right[a], b)
        upd(a)
        return a
    else:
        left[b] = merge(a, left[b])
        upd(b)
        return b

def insert(root, k):
    a, b = split(root, k)
    node = new_node(k)
    return merge(a, merge(node, b))

def erase(root, k):
    a, b = split(root, k)
    c, d = split(b, k + 1)
    return merge(a, d)

def kth_max(root, k):
    v = root
    while True:
        r = sz(right[v])
        if k == r + 1:
            return key[v]
        elif k <= r:
            v = right[v]
        else:
            k -= r + 1
            v = left[v]

root = -1
out = []

for _ in range(n):
    c = int(data[pos]); pos += 1
    k = int(data[pos]); pos += 1
    if c == 1:
        root = insert(root, k)
    elif c == -1:
        root = erase(root, k)
    else:
        out.append(str(kth_max(root, k)))

open("output.txt", "w").write("\n".join(out))
```

```
main()
```

Тесты

input.txt

1	5
2	1 5
3	1 3
4	1 7
5	0 1
6	0 3

output.txt

1	7
2	3

Тест 1

input.txt

1	5
2	1 5
3	1 3
4	1 7
5	-1 7
6	0 1

output.txt

1	5

Тест 2

input.txt

1	4
2	1 10
3	-1 10
4	1 20
5	0 1

output.txt

1	20

Tecm 3